

Recursion

Function calling itself is called recursion

A recursion method solves a problem by calling a copy of itself to work on a smaller problem.

Each time the function call itself with a slightly simpler version of the ~~the~~ original problem.

Recursive Tree

```
int main()
```

```
{
```

```
    int k;
```

```
    k = f1(8);
```

```
    printf("%d", k);
```

```
}
```

Not a recursive function

```
int f1(int n)
```

```
{
```

```
    int s;
```

```
    if (n == 1)
```

```
        return 1;
```

```
    s = n + f1(n-1);
```

```
    return (s);
```

```
}
```

Recursive function

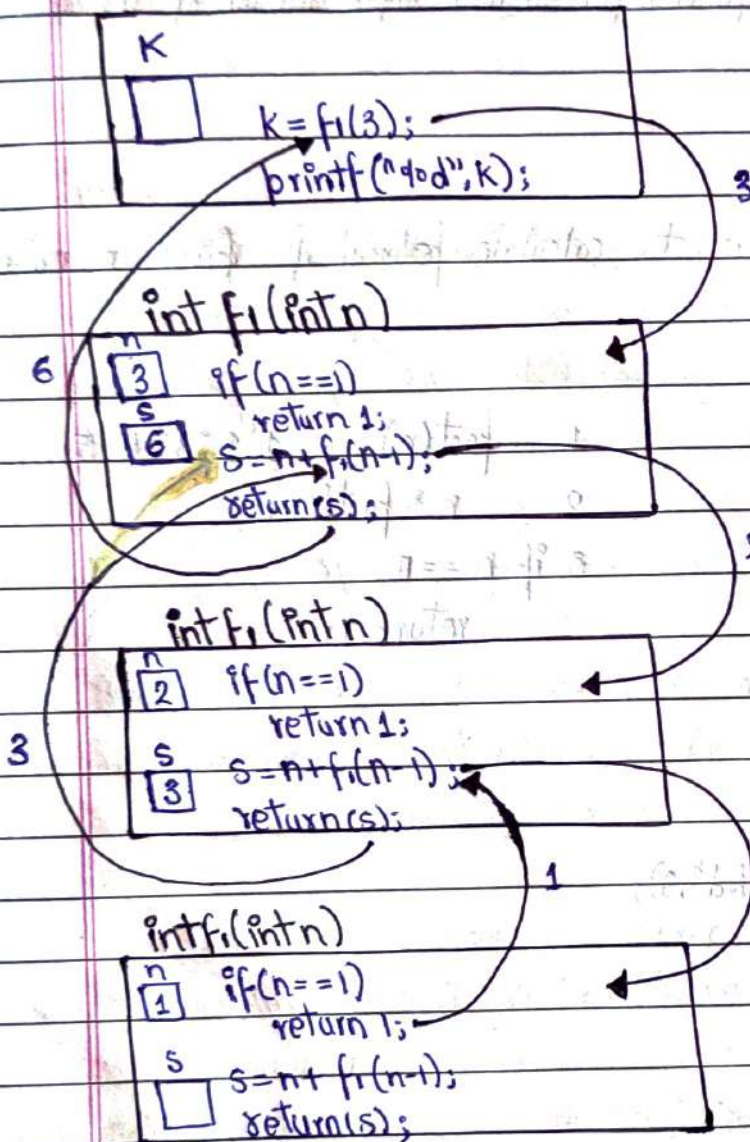
function call itself to solve slightly simpler version of the original problem

→ 2 function defined hai

Recursion Tree

main()

$$* f_1(n) = n + f_1(n-1)$$





How to approach a Recursive problem?

1. Suppose funcn is made
2. Recursive case $\rightarrow$ calling same function for slightly simple version of the original prob
3. Base case

Q $\rightarrow$ Write a recursive function to calculate factorial of n .

1. $\text{fact}(n) = 1 * 2 * 3 * 4 * 5 \dots n$
2. $n * \text{fact}(n-1)$
3. if $n == 0$
return 1;

```
#include <stdio.h>
```

```
int fact(int m)
```

```
int main()
```

```
{ int n, s;
```

```
printf("Enter the value of n = ");
```

```
scanf("%d", &n);
```

```
s = fact(n)
```

```
printf("factorial of given number = %d", s);
```

```
return 0;
```

```
}
```

```
int fact(int m)
```

```
{
```

```
if (m == 0)
```

```
return 1;
```

```
return (m * fact(m-1));
```

```
}
```